

Terceros pasos en Python

Pensamiento Computacional

Facultad de Ciencias Exactas y Naturales, UBA

Segundo Cuatrimestre 2026

RECAPITULANDO

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número repeticiones a hacer, genero lista para guardar cantidad de figus compradas cada vez.

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número `repeticiones` a hacer, genero lista para guardar cantidad de figus compradas cada vez.
- `paso`: cuenta en qué repetición estoy.

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número repeticiones a hacer, genero lista para guardar cantidad de figus compradas cada vez.
- paso: cuenta en qué repetición estoy.
- Indico el tamaño del álbum a llenar.

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número repeticiones a hacer, genero lista para guardar cantidad de figus compradas cada vez.
- paso: cuenta en qué repetición estoy.
- Indico el tamaño del álbum a llenar.
- Ciclo hasta completar las repeticiones.

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número `repeticiones` a hacer, genero lista para guardar cantidad de figus compradas cada vez.
- `paso`: cuenta en qué repetición estoy.
- Indico el tamaño del álbum a llenar.
- Ciclo hasta completar las repeticiones.
- En cada paso defino nuevo album y reseteo `cant_figus` a 0.

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número repeticiones a hacer, genero lista para guardar cantidad de figus compradas cada vez.
- paso: cuenta en qué repetición estoy.
- Indico el tamaño del álbum a llenar.
- Ciclo hasta completar las repeticiones.
- En cada paso defino nuevo album y reseteo cant_figus a 0.
- Hago un ciclo interior para simular el llenado.

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número repeticiones a hacer, genero lista para guardar cantidad de figus compradas cada vez.
- paso: cuenta en qué repetición estoy.
- Indico el tamaño del álbum a llenar.
- Ciclo hasta completar las repeticiones.
- En cada paso defino nuevo album y reseteo cant_figus a 0.
- Hago un ciclo interior para simular el llenado.
- Almaceno la cantidad de figuritas compradas.

Teníamos el siguiente código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

- Defino el número `repeticiones` a hacer, genero lista para guardar cantidad de figus compradas cada vez.
- `paso`: cuenta en qué repetición estoy.
- Indico el tamaño del álbum a llenar.
- Ciclo hasta completar las repeticiones.
- En cada paso defino nuevo album y reseteo `cant_figus` a 0.
- Hago un ciclo interior para simular el llenado.
- Almaceno la cantidad de figuritas compradas.
- Calculo e imprimo la cantidad promedio.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.
 - Repite la simulación muchas veces.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.
 - Repite la simulación muchas veces.
 - Llena una lista con los resultados de la simulación.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.
 - Repite la simulación muchas veces.
 - Llena una lista con los resultados de la simulación.
 - Calcula el promedio de esa lista y lo imprime.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.
 - Repite la simulación muchas veces.
 - Llena una lista con los resultados de la simulación.
 - Calcula el promedio de esa lista y lo imprime.
- En el camino, define muchas variables y es importante recordar qué función cumple cada una para no meter la pata.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.
 - Repite la simulación muchas veces.
 - Llena una lista con los resultados de la simulación.
 - Calcula el promedio de esa lista y lo imprime.
- En el camino, define muchas variables y es importante recordar qué función cumple cada una para no meter la pata.
- Algunas variables (`repeticiones`, `tam_album`) tienen valores que son datos generales del problema que me gustaría poder cambiar.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.
 - Repite la simulación muchas veces.
 - Llena una lista con los resultados de la simulación.
 - Calcula el promedio de esa lista y lo imprime.
- En el camino, define muchas variables y es importante recordar qué función cumple cada una para no meter la pata.
- Algunas variables (`repeticiones`, `tam_album`) tienen valores que son datos generales del problema que me gustaría poder cambiar.
- Si queremos hacer más cosas (calcular chances de ganar con cierta cierta cantidad de figus compradas, calcular el máximo de figus a comprar, graficar, etc.), el código se va a complejizar aún más.

Problemas

Esto que produjimos es lo que llamamos un **script**: es decir un archivo de texto que contiene código de manera que al ejecutarlo `Python` efectivamente realiza un montón de operaciones que nos devuelven algo. Pero tiene algunos problemas:

- El código es largo.
- Hace **muchas** cosas:
 - Simula un llenado del álbum.
 - Repite la simulación muchas veces.
 - Llena una lista con los resultados de la simulación.
 - Calcula el promedio de esa lista y lo imprime.
- En el camino, define muchas variables y es importante recordar qué función cumple cada una para no meter la pata.
- Algunas variables (`repeticiones`, `tam_album`) tienen valores que son datos generales del problema que me gustaría poder cambiar.
- Si queremos hacer más cosas (calcular chances de ganar con cierta cierta cantidad de figus compradas, calcular el máximo de figus a comprar, graficar, etc.), el código se va a complicar aún más.
- Este código es difícil de *compartir*: si se lo damos a alguien para que lo pruebe con su propio álbum (otras repeticiones, otra cantidad de figus, etc.), esa otra persona lo tiene que entender para poder usarlo.

FUNCIONES

Funciones

¿Qué es una función en programación?

Es un **bloque de código** que realiza una función específica, que tiene un nombre que nos permite utilizarla.

Funciones

¿Qué es una función en programación?

Es un **bloque de código** que realiza una función específica, que tiene un nombre que nos permite utilizarla.

Ejemplos:

- `len`: *recibe* una lista y *devuelve* su longitud.

Funciones

¿Qué es una función en programación?

Es un **bloque de código** que realiza una función específica, que tiene un nombre que nos permite utilizarla.

Ejemplos:

- `len`: *recibe* una lista y *devuelve* su longitud.
- `sum`: *recibe* una lista y *devuelve* la suma de sus elementos.

Funciones

¿Qué es una función en programación?

Es un **bloque de código** que realiza una función específica, que tiene un nombre que nos permite utilizarla.

Ejemplos:

- `len`: *recibe* una lista y *devuelve* su longitud.
- `sum`: *recibe* una lista y *devuelve* la suma de sus elementos.
- `random.random()`: *no recibe nada* y *devuelve* un número al azar entre 0 y 1.

Funciones

¿Qué es una función en programación?

Es un **bloque de código** que realiza una función específica, que tiene un nombre que nos permite utilizarla.

Ejemplos:

- `len`: *recibe* una lista y *devuelve* su longitud.
- `sum`: *recibe* una lista y *devuelve* la suma de sus elementos.
- `random.random()`: *no recibe nada* y *devuelve* un número al azar entre 0 y 1.
- `random.randint(a, b)`: *recibe* dos valores enteros *a* y *b* y *devuelve* un número entero elegido al azar, entre *a* y *b* (incluyendo los extremos).

Funciones

¿Qué es una función en programación?

Es un **bloque de código** que realiza una función específica, que tiene un nombre que nos permite utilizarla.

Ejemplos:

- `len`: *recibe* una lista y *devuelve* su longitud.
- `sum`: *recibe* una lista y *devuelve* la suma de sus elementos.
- `random.random()`: *no recibe nada* y *devuelve* un número al azar entre 0 y 1.
- `random.randint(a, b)`: *recibe* dos valores enteros *a* y *b* y *devuelve* un número entero elegido al azar, entre *a* y *b* (incluyendo los extremos).
- `random.shuffle(lista)`: *recibe* una lista y *no devuelve nada*, sino que la desordena al azar.

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a,b)`? ¡Ni idea!

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a,b)`? ¡Ni idea!
- Sólo necesitamos saber qué espera *recibir* una función y qué nos va a *devolver*.

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a,b)`? ¡Ni idea!
- Sólo necesitamos saber qué espera *recibir* una función y qué nos va a *devolver*.

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a,b)`? ¡Ni idea!
- Sólo necesitamos saber qué espera *recibir* una función y qué nos va a *devolver*.

En principio las funciones pueden pensarse, ingenuamente, como funciones matemáticas:

$$f(x) = 2x + 1$$

- Es la definición matemática de una función que se llama f .

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a,b)`? ¡Ni idea!
- Sólo necesitamos saber qué espera *recibir* una función y qué nos va a *devolver*.

En principio las funciones pueden pensarse, ingenuamente, como funciones matemáticas:

$$f(x) = 2x + 1$$

- Es la definición matemática de una función que se llama f .
- f recibe un número y devuelve el resultado de multiplicarlo por 2 y sumarle 1.

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a, b)`? ¡Ni idea!
- Sólo necesitamos saber qué espera *recibir* una función y qué nos va a *devolver*.

En principio las funciones pueden pensarse, ingenuamente, como funciones matemáticas:

$$f(x) = 2x + 1$$

- Es la definición matemática de una función que se llama f .
- f recibe un número y devuelve el resultado de multiplicarlo por 2 y sumarle 1.
- x es sólo un nombre genérico que nos permite escribir una fórmula para la función.

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a,b)`? ¡Ni idea!
- Sólo necesitamos saber qué espera *recibir* una función y qué nos va a *devolver*.

En principio las funciones pueden pensarse, ingenuamente, como funciones matemáticas:

$$f(x) = 2x + 1$$

- Es la definición matemática de una función que se llama f .
- f recibe un número y devuelve el resultado de multiplicarlo por 2 y sumarle 1.
- x es sólo un nombre genérico que nos permite escribir una fórmula para la función.
- Para *usar* f necesitamos reemplazar x por un número concreto:
 $f(3) = 2 \cdot 3 + 1 = 7$.

Como usuarios, las funciones nos ayudan porque:

- Nos permiten realizar operaciones complejas sin necesidad de programarlas nosotros.
- Ni siquiera necesitamos comprender el código de la función para usarla: ¿qué hace internamente la función `random.randint(a,b)`? ¡Ni idea!
- Sólo necesitamos saber qué espera *recibir* una función y qué nos va a *devolver*.

En principio las funciones pueden pensarse, ingenuamente, como funciones matemáticas:

$$f(x) = 2x + 1$$

- Es la definición matemática de una función que se llama f .
- f recibe un número y devuelve el resultado de multiplicarlo por 2 y sumarle 1.
- x es sólo un nombre genérico que nos permite escribir una fórmula para la función.
- Para *usar* f necesitamos reemplazar x por un número concreto:
 $f(3) = 2 \cdot 3 + 1 = 7$.
- Decimos que la función f *evaluada* en 3 *devuelve* 7.

Similitudes

Ejemplo:

```
random.randint(a,b)
```

Similitudes

Ejemplo:

```
random.randint(a,b)
```

Recibe dos números, hace una serie de operaciones (para nosotros desconocidas) usando esos números y con eso produce un resultado: un número entero al azar entre a y b .

- No conocemos el código (la *fórmula*) de la función, pero seguramente es bastante más complejo que la formulita de f .

Similitudes

Ejemplo:

```
random.randint(a,b)
```

Recibe dos números, hace una serie de operaciones (para nosotros desconocidas) usando esos números y con eso produce un resultado: un número entero al azar entre a y b .

- No conocemos el código (la *fórmula*) de la función, pero seguramente es bastante más complejo que la formulita de f .
- El código de la función, escrito, no calcula nada, del mismo modo en que $2x + 1$ no calcula nada. Hay que proveer un valor de x .

Similitudes

Ejemplo:

```
random.randint(a,b)
```

Recibe dos números, hace una serie de operaciones (para nosotros desconocidas) usando esos números y con eso produce un resultado: un número entero al azar entre a y b .

- No conocemos el código (la *fórmula*) de la función, pero seguramente es bastante más complejo que la formulita de f .
- El código de la función, escrito, no calcula nada, del mismo modo en que $2x + 1$ no calcula nada. Hay que proveer un valor de x .
- Recién cuando damos valores para a y b la máquina puede *ejecutar* la función y devolver algo.

Diferencias

Sin embargo, las funciones en programación puede ser **muy distintas** a las funciones matemáticas. En programación

- La *fórmula* suele ser una compleja secuencia de pasos, y no sólo una operación matemática.

Diferencias

Sin embargo, las funciones en programación puede ser **muy distintas** a las funciones matemáticas. En programación

- La *fórmula* suele ser una compleja secuencia de pasos, y no sólo una operación matemática.
- Hay funciones que *modifican* lo que reciben: `random.shuffle()` recibe una lista y la *mezcla*.

Diferencias

Sin embargo, las funciones en programación puede ser **muy distintas** a las funciones matemáticas. En programación

- La *fórmula* suele ser una compleja secuencia de pasos, y no sólo una operación matemática.
- Hay funciones que *modifican* lo que reciben: `random.shuffle()` recibe una lista y la *mezcla*.
- Hay funciones que *no devuelven nada* (también `random.shuffle()`).

Diferencias

Sin embargo, las funciones en programación puede ser **muy distintas** a las funciones matemáticas. En programación

- La *fórmula* suele ser una compleja secuencia de pasos, y no sólo una operación matemática.
- Hay funciones que *modifican* lo que reciben: `random.shuffle()` recibe una lista y la *mezcla*.
- Hay funciones que *no devuelven nada* (también `random.shuffle()`).
- Una función puede recibir varias cosas, modificar algunas de ellas y otras no, y devolver varias otras cosas.

Diferencias

Sin embargo, las funciones en programación puede ser **muy distintas** a las funciones matemáticas. En programación

- La *fórmula* suele ser una compleja secuencia de pasos, y no sólo una operación matemática.
- Hay funciones que *modifican* lo que reciben: `random.shuffle()` recibe una lista y la *mezcla*.
- Hay funciones que *no devuelven nada* (también `random.shuffle()`).
- Una función puede recibir varias cosas, modificar algunas de ellas y otras no, y devolver varias otras cosas.
- Una función puede *no recibir nada* e igual realizar una operación y devolver algo, como `random.random()`.

Programando funciones

Para códigos simples no hace falta definir funciones (ya resolvimos un problema interesante, sin hacerlo).

Programando funciones

Para códigos simples no hace falta definir funciones (ya resolvimos un problema interesante, sin hacerlo).

Sin embargo, escribir funciones propias permite:

- Ordenar al código.

Programando funciones

Para códigos simples no hace falta definir funciones (ya resolvimos un problema interesante, sin hacerlo).

Sin embargo, escribir funciones propias permite:

- Ordenar al código.
- Reutilizar partes de código sin tener que copiarlas.

Programando funciones

Para códigos simples no hace falta definir funciones (ya resolvimos un problema interesante, sin hacerlo).

Sin embargo, escribir funciones propias permite:

- Ordenar al código.
- Reutilizar partes de código sin tener que copiarlas.
- Liberarnos de tener que recordar el significado de todas los nombres que pusimos a las variables.

Programando funciones

Para códigos simples no hace falta definir funciones (ya resolvimos un problema interesante, sin hacerlo).

Sin embargo, escribir funciones propias permite:

- Ordenar al código.
- Reutilizar partes de código sin tener que copiarlas.
- Liberarnos de tener que recordar el significado de todas los nombres que pusimos a las variables.
- Compartir nuestros programas para que otros los usen sin saber (ni entender), el código que escribimos: del mismo modo que nosotros usamos las funciones de `Python` sin saber exactamente cómo hacen lo que hacen.

Es importante decidir qué código es **razonable** poner dentro de una función.
Típicamente:

- Si un trozo de código hace una operación con sentido.
- Si puede ser útil repetir esa operación en otros lugares y/o a partir de datos diferentes.

entonces es conveniente encapsular ese código dentro de una función.

Es importante decidir qué código es **razonable** poner dentro de una función. Típicamente:

- Si un trozo de código hace una operación con sentido.
- Si puede ser útil repetir esa operación en otros lugares y/o a partir de datos diferentes.

entonces es conveniente encapsular ese código dentro de una función. Además,

- Una función tiene que tener un comportamiento definido claro.
- Se debería poder explicar qué hace de manera sencilla.

Consideremos este código:

```
import random

repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
paso = 0
while paso < repeticiones:
    album = [0]*tam_album
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam_album-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    cantidades[paso] = cant_figus
    paso = paso + 1
promedio = sum(cantidades)/len(cantidades)
print("El promedio es:", promedio)
```

Consideremos este código:

```
import random
```

```
repeticiones = 1000  
cantidades = [0]*repeticiones  
tam_album = 6
```

Variables
generales

```
paso = 0
```

```
while paso < repeticiones:
```

← Repite muchas veces

```
    album = [0]*tam_album
```

```
    cant_figus = 0
```

```
    while sum(album)<len(album):
```

```
        figu = random.randint(0,tam_album-1)
```

```
        album[figu] = 1
```

```
        cant_figus = cant_figus + 1
```

} Simula el llenado
de un album

```
    cantidades[paso] = cant_figus
```

```
    paso = paso + 1
```

```
promedio = sum(cantidades)/len(cantidades)
```

```
print("El promedio es:", promedio)
```

← Almacena
el resultado

Consideremos este código:

```
import random
```

```
repeticiones = 1000
cantidades = [0]*repeticiones
tam_album = 6
```

Variables
generales

```
paso = 0
```

```
while paso < repeticiones:
```

Repite muchas veces

```
    album = [0]*tam_album
```

```
    cant_figur = 0
```

```
    while sum(album)<len(album):
```

```
        figur = random.randint(0,tam_album-1)
```

```
        album[figur] = 1
```

```
        cant_figur = cant_figur + 1
```

Simula el llenado
de un album

```
    cantidades[paso] = cant_figur
```

```
    paso = paso + 1
```

Almacena
el resultado

```
promedio = sum(cantidades)/len(cantidades)
```

```
print("El promedio es:", promedio)
```

Tener un **ciclo anidado**, usualmente es una pista sólida respecto a la necesidad de definir una función. El código marcado por la llave es candidato **razonable** para función.

Funciones

El código de una función se puede dividir en tres partes:

- 1 El **encabezado** o **firma**: acá se indica el nombre de la función y qué *argumentos* recibe.

Funciones

El código de una función se puede dividir en tres partes:

- 1 El **encabezado** o **firma**: acá se indica el nombre de la función y qué *argumentos* recibe.
- 2 El **cuerpo**: acá va el código que hace las operaciones para producir el resultado deseado.

Funciones

El código de una función se puede dividir en tres partes:

- 1 El **encabezado** o **firma**: acá se indica el nombre de la función y qué *argumentos* recibe.
- 2 El **cuerpo**: acá va el código que hace las operaciones para producir el resultado deseado.
- 3 La **salida**: acá se indica qué es lo que la función devuelve. Es opcional: puede no devolver nada.

Sintaxis

Sintaxis

```
def nombre_funcion(argumento1, argumento2, ...):  
    #  
    codigo  
    #  
    return ...
```

Sintaxis

```
def nombre_funcion(argumento1, argumento2, ...):  
    #  
    codigo  
    #  
    return ...
```

- La palabra clave **def** indica que se define una función.

Sintaxis

```
def nombre_funcion(argumento1, argumento2, ...):  
    #  
    codigo  
    #  
    return ...
```

- La palabra clave **def** indica que se define una función.
- `nombre_funcion` es el nombre de la función. Puede ser más o menos cualquier cosa (valen las mismas reglas que para las variables).

Sintaxis

```
def nombre_funcion(argumento1, argumento2, ...):  
    #  
    codigo  
    #  
    return ...
```

- La palabra clave **def** indica que se define una función.
- `nombre_funcion` es el nombre de la función. Puede ser más o menos cualquier cosa (valen las mismas reglas que para las variables).
- Entre paréntesis se indican todos los **argumentos** que espera recibir la función.

Sintaxis

```
def nombre_funcion(argumento1, argumento2, ...):  
    #  
    codigo  
    #  
    return ...
```

- La palabra clave **def** indica que se define una función.
- `nombre_funcion` es el nombre de la función. Puede ser más o menos cualquier cosa (valen las mismas reglas que para las variables).
- Entre paréntesis se indican todos los **argumentos** que espera recibir la función.
- El **return** es opcional y sirve para indicar qué valor debe devolver la función.

Sintaxis

```
def nombre_funcion(argumento1, argumento2, ...):  
    #  
    codigo  
    #  
    return ...
```

- La palabra clave **def** indica que se define una función.
- `nombre_funcion` es el nombre de la función. Puede ser más o menos cualquier cosa (valen las mismas reglas que para las variables).
- Entre paréntesis se indican todos los **argumentos** que espera recibir la función.
- El **return** es opcional y sirve para indicar qué valor debe devolver la función.
- Típicamente, el código interno genera el valor que queremos devolver.

Sintaxis

```
def nombre_funcion(argumento1, argumento2, ...):  
    #  
    codigo  
    #  
    return ...
```

- La palabra clave **def** indica que se define una función.
- `nombre_funcion` es el nombre de la función. Puede ser más o menos cualquier cosa (valen las mismas reglas que para las variables).
- Entre paréntesis se indican todos los **argumentos** que espera recibir la función.
- El **return** es opcional y sirve para indicar qué valor debe devolver la función.
- Típicamente, el código interno genera el valor que queremos devolver.
- El código interno de la función, incluido el **return** queda demarcado por la tabulación.

Primera función

¿Qué datos debe recibir nuestro código para simular un llenado de álbum?

Primera función

¿Qué datos debe recibir nuestro código para simular un llenado de álbum?
El **tamaño del álbum**. Y nada más.

Primera función

¿Qué datos debe recibir nuestro código para simular un llenado de álbum?

El **tamaño del álbum**. Y nada más.

¿Qué debe devolver nuestra función?

Primera función

¿Qué datos debe recibir nuestro código para simular un llenado de álbum?

El **tamaño del álbum**. Y nada más.

¿Qué debe devolver nuestra función?

La **cantidad de figus** que hubo que comprar para llenar el álbum.

Primera función

¿Qué datos debe recibir nuestro código para simular un llenado de álbum?

El **tamaño del álbum**. Y nada más.

¿Qué debe devolver nuestra función?

La **cantidad de figus** que hubo que comprar para llenar el álbum.

```
def cantidad_para_llenar(tam):  
    album = [0]*tam  
    cant_figus = 0  
    while sum(album) < len(album):  
        figu = random.randint(0,tam-1)  
        album[figu] = 1  
        cant_figus = cant_figus + 1  
    return cant_figus
```

Con esto nuestro código podría quedar así:

```
import random

def cantidad_para_llenar(tam):
    album = [0]*tam
    cant_figus = 0
    while sum(album)<len(album):
        figu = random.randint(0,tam-1)
        album[figu] = 1
        cant_figus = cant_figus + 1
    return cant_figus

repeticiones = 1000
cantidades = [0]*repeticiones
paso = 0
tam_album = 6
while paso < repeticiones:
    cantidades[paso] = cantidad_para_llenar(tam_album)
    paso = paso + 1

promedio = sum(cantidades)/len(cantidades)
print(promedio)
```

Experimentemos en Python-tutor

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.
- El **argumento** `tam_album` en la definición es como la x en la definición de $f(x)$: representa un valor que la función espera.

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.
- El **argumento** `tam_album` en la definición es como la x en la definición de $f(x)$: representa un valor que la función espera.
- La función **asume** que va a recibir un valor e internamente llama a ese valor `tam_album`

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.
- El **argumento** `tam_album` en la definición es como la x en la definición de $f(x)$: representa un valor que la función espera.
- La función **asume** que va a recibir un valor e internamente llama a ese valor `tam_album`
- Para ejecutar la función es necesario escribir su nombre y, entre paréntesis, los valores de los argumentos esperados. En nuestro caso:
`cantidad_para_llenar(tam_album)`. Al hacer esto, se crea la variable interna `tam` y se le asigna el valor guardado en la variable externa `tam_album`.

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.
- El **argumento** `tam_album` en la definición es como la x en la definición de $f(x)$: representa un valor que la función espera.
- La función **asume** que va a recibir un valor e internamente llama a ese valor `tam_album`
- Para ejecutar la función es necesario escribir su nombre y, entre paréntesis, los valores de los argumentos esperados. En nuestro caso:
`cantidad_para_llenar(tam_album)`. Al hacer esto, se crea la variable interna `tam` y se le asigna el valor guardado en la variable externa `tam_album`.
- Las variables *internas* de la función (`tam`, `album`, `cant_figus`, `figu`) **desaparecen** cuando la función termina su ejecución. Y se vuelven a crear, cada vez que la función se corre.

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.
- El **argumento** `tam_album` en la definición es como la x en la definición de $f(x)$: representa un valor que la función espera.
- La función **asume** que va a recibir un valor e internamente llama a ese valor `tam_album`
- Para ejecutar la función es necesario escribir su nombre y, entre paréntesis, los valores de los argumentos esperados. En nuestro caso:
`cantidad_para_llenar(tam_album)`. Al hacer esto, se crea la variable interna `tam` y se le asigna el valor guardado en la variable externa `tam_album`.
- Las variables *internas* de la función (`tam`, `album`, `cant_figus`, `figu`) **desaparecen** cuando la función termina su ejecución. Y se vuelven a crear, cada vez que la función se corre.
- El **valor** de la variable interna `cant_figuritas` se devuelve y se puede almacenar en una variable. Pero su nombre interno desaparece.

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.
- El **argumento** `tam_album` en la definición es como la x en la definición de $f(x)$: representa un valor que la función espera.
- La función **asume** que va a recibir un valor e internamente llama a ese valor `tam_album`
- Para ejecutar la función es necesario escribir su nombre y, entre paréntesis, los valores de los argumentos esperados. En nuestro caso:
`cantidad_para_llenar(tam_album)`. Al hacer esto, se crea la variable interna `tam` y se le asigna el valor guardado en la variable externa `tam_album`.
- Las variables *internas* de la función (`tam`, `album`, `cant_figus`, `figu`) **desaparecen** cuando la función termina su ejecución. Y se vuelven a crear, cada vez que la función se corre.
- El **valor** de la variable interna `cant_figuritas` se devuelve y se puede almacenar en una variable. Pero su nombre interno desaparece.
- Como **usuarios** de la función no necesitamos saber ni los nombres que usa el código para los **argumentos** (`tam`) ni los nombres de las demás variables internas, ni los detalles del procedimiento.

Conclusiones importante

- La definición de la función por si misma **no hace nada**: es necesario **ejecutar** la función con datos concretos.
- El **argumento** `tam_album` en la definición es como la x en la definición de $f(x)$: representa un valor que la función espera.
- La función **asume** que va a recibir un valor e internamente llama a ese valor `tam_album`
- Para ejecutar la función es necesario escribir su nombre y, entre paréntesis, los valores de los argumentos esperados. En nuestro caso:
`cantidad_para_llenar(tam_album)`. Al hacer esto, se crea la variable interna `tam` y se le asigna el valor guardado en la variable externa `tam_album`.
- Las variables *internas* de la función (`tam`, `album`, `cant_figus`, `figu`) **desaparecen** cuando la función termina su ejecución. Y se vuelven a crear, cada vez que la función se corre.
- El **valor** de la variable interna `cant_figuritas` se devuelve y se puede almacenar en una variable. Pero su nombre interno desaparece.
- Como **usuarios** de la función no necesitamos saber ni los nombres que usa el código para los **argumentos** (`tam`) ni los nombres de las demás variables internas, ni los detalles del procedimiento.
- Como **usuarios** de la función alcanza con saber: el nombre de la función, qué tipo de argumentos recibe (cuántos son, si son números o listas, etc.), y qué devuelve.

Receta

Siempre que programemos una función **debemos**:

- Verificar que todas las variables estén definidas **en** la función. Es decir:

Receta

Siempre que programemos una función **debemos**:

- Verificar que todas las variables estén definidas **en** la función. Es decir:
 - Que sean **argumentos** que la función espera recibir para poder ejecutarse.

Receta

Siempre que programemos una función **debemos**:

- Verificar que todas las variables estén definidas **en** la función. Es decir:
 - Que sean **argumentos** que la función espera recibir para poder ejecutarse.
 - O bien, que estén definidas **internamente**.

Receta

Siempre que programemos una función **debemos**:

- Verificar que todas las variables estén definidas **en** la función. Es decir:
 - Que sean **argumentos** que la función espera recibir para poder ejecutarse.
 - O bien, que estén definidas **internamente**.
- **Probar** la función:

Receta

Siempre que programemos una función **debemos**:

- Verificar que todas las variables estén definidas **en** la función. Es decir:
 - Que sean **argumentos** que la función espera recibir para poder ejecutarse.
 - O bien, que estén definidas **internamente**.
- **Probar** la función:
 - Es esencial ejecutar la función en casos de ejemplo y constatar que devuelve lo que esperamos.

Receta

Siempre que programemos una función **debemos**:

- Verificar que todas las variables estén definidas **en** la función. Es decir:
 - Que sean **argumentos** que la función espera recibir para poder ejecutarse.
 - O bien, que estén definidas **internamente**.
- **Probar** la función:
 - Es esencial ejecutar la función en casos de ejemplo y constatar que devuelve lo que esperamos.
 - Es importante probar distintos casos, para asegurarse de que la función hace lo esperado siempre y no sólo en una situación puntual.

Esto recién comienza. . .

Terminamos. . . ¡sólo por hoy!